

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Programming Integration Project (Semester 252)

Assignment

**Boolean Network Analyzer –
Parsing, Simulation and
Visualisation of Boolean Networks**

Instructor(s): Trinh Van Giang

Student(s): Tran Phan Dang Khoi 2352626

HO CHI MINH CITY, MAY 2026

Abstract

Project (Topic 6): BOOLEAN NETWORK ANALYZER – a software tool for parsing, simulating and visualising Boolean Networks.

A Boolean Network (BN) is a discrete mathematical model originally proposed by Stuart Kauffman in 1969 to study *gene regulatory networks*. Each node in the network carries one of two values $\{0, 1\}$ and is updated by a Boolean function of its neighbours' states. In spite of its very simple representation, a Boolean Network is expressive enough to describe *steady states*, *cyclic oscillations* and *complex* behaviours of real biological and industrial logic systems. Tools such as GINSIM, PYBOOLNET, BOOLNET (R) and CELL-NETANALYZER have become de facto international standards for BN dynamics research.

This report documents the *theoretical study, architectural design, implementation and evaluation* of a complete BOOLEAN NETWORK ANALYZER delivered as a zero-install, in-browser web application hosted on GitHub Pages¹. The software supports five mandatory feature blocks: (i) a **Parser** that reads the standard `.bnet` format (compatible with PyBoolNet) consisting of update rules of the form `target, expression`; (ii) an **Influence Graph Builder** that constructs the signed influence graph (+/-/?), with the sign inferred from a monotonicity test on the Boolean function; (iii) a **State Transition Graph (STG) Engine** that enumerates the entire 2^n state space under the two canonical update schemes – *synchronous* and *asynchronous*; (iv) an **Attractor Finder** that applies strongly connected component (SCC) decomposition (Tarjan) on the condensation graph to identify every attractor and classify it as *fixed-point*, *cyclic* or *complex*; (v) a **Visualiser** that produces interactive renderings of all graphs in the browser (using `vis-network`), with PNG export for reporting. The same algorithms are also exposed as a Python reference implementation used by the automated test suite.

The project offers four main contributions: **(1) A clean modular architecture** (Parser $\rightarrow$ Network $\rightarrow$ STG $\rightarrow$ Attractors $\rightarrow$ Visualizer) that separates I/O from algorithmic logic, implemented twice in parallel (Python reference + vanilla-JavaScript in-browser engine) and cross-validated against each other; **(2) An automated test suite (pytest)** covering parser, network evaluation, STG, attractors and bundled samples; **(3) Cross-validation against the reference tool PyBoolNet** on five sample networks (*toy switch*, *bistable toggle*, *repressilator*, *fission yeast*, *mammalian cell cycle*) – the attractor results match 100%; **(4) A friendly zero-install web frontend** for

¹Source: <https://github.com/dangkhoi-dev/boolean-network-analyzer>. Live demo: <https://dangkhoi-dev.github.io/boolean-network-analyzer/>



non-programmers: a single static HTML/JS page deployed via GitHub Pages, integrating file upload, sample selection, scheme switching, trajectory simulation and figure download with no backend or installation required.

The software has been tested with five networks ranging from 2 to 10 nodes. In every case the attractor results agree with the original publications (Davidich & Bornholdt 2008, Faure *et al.* 2006, Gardner *et al.* 2000). Performance evaluation shows that the software is responsive for networks with ≤ 16 nodes (i.e. up to $2^{16} = 65\,536$ states); beyond that threshold the cost of generating the full STG grows exponentially and the user is encouraged to switch to *reachable-only* mode (BFS from a chosen initial state). All source code, sample dataset and documentation are organised according to a professional project layout (`src/`, `tests/`, `samples/`, `docs/`) and released on a public GitHub repository.

Keywords: Boolean Network, Influence Graph, State Transition Graph, Attractor, Synchronous Update, Asynchronous Update, Strongly Connected Component, PyBoolNet, GitHub Pages, vis-network, Bioinformatics.



Contents

Abstract	i
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Background	1
1.1.1 Boolean Networks in systems science	1
1.1.2 Applications of Boolean Networks	1
1.1.3 Tooling gap and motivation	2
1.2 Project objectives	2
1.3 Project scope	3
1.4 Main contributions	4
1.5 Report structure	5
2 Theoretical Background	5
2.1 Mathematical definition of a Boolean Network	6
2.1.1 Formal definition	6
2.1.2 The <code>.bnet</code> rule format	6
2.1.3 Illustrative example: a 3-node <i>toy switch</i>	6
2.2 The influence graph	7
2.2.1 Definition	7
2.2.2 Inferring the edge sign	8
2.2.3 Worked example	8
2.2.4 Role of the influence graph	9
2.3 Update schemes: synchronous vs asynchronous	9
2.3.1 Synchronous scheme	10
2.3.2 Asynchronous scheme	10
2.3.3 A side-by-side comparison: the 2-node bistable toggle	11
2.4 The State Transition Graph (STG)	12
2.4.1 Definition	12
2.4.2 Size and representation	12
2.4.3 Example: synchronous STG of the <i>repressilator</i>	13
2.5 Attractors: point, cyclic and complex	14
2.5.1 Definition	14



2.5.2	Classification of attractors	14
2.5.3	Basin of attraction	15
2.5.4	Biological and engineering meaning	15
2.6	Comparison with reference tools	16
3	Methodology & System Design	16
3.1	Overall architecture	16
3.2	Source code layout	18
3.3	In-memory data structures	20
3.3.1	ParsedRule	20
3.3.2	State	20
3.3.3	Graphs: NetworkX DiGraph	21
3.4	Core algorithms	21
3.4.1	Parser	21
3.4.2	Evaluation	22
3.4.3	Building the influence graph	22
3.4.4	Building the STG	22
3.4.5	Finding attractors: SCC + condensation	23
3.4.6	Computing the basin of attraction	24
3.5	User interface design (in-browser web app)	24
3.6	Methodology summary	25
4	Experimental Results & Evaluation	26
4.1	Sample networks	26
4.2	Per-test results	27
4.2.1	Test case 1: <code>toy_switch</code> (3 nodes)	27
4.2.2	Test case 2: <code>bistable_toggle</code> (2 nodes)	27
4.2.3	Test case 3: <code>repressilator</code> (3 nodes)	28
4.2.4	Test case 4: <code>fission_yeast</code> (9 nodes)	28
4.2.5	Test case 5: <code>mammalian_cell_cycle</code> (10 nodes)	29
4.3	Correctness validation	29
4.3.1	Validation against PyBoolNet	29
4.3.2	Validation against manual computation	30
4.3.3	Automated test suite (pytest)	31
4.4	Performance analysis	32
4.5	Bugs found and fixed during development	33
4.6	Experimental summary	33



5 Conclusion & Future Work	33
5.1 Summary	33
5.2 Limitations	35
5.3 Future work	35
5.4 Acknowledgements	36
References	37



List of Figures

2.1	Influence graph of the 3-node <i>toy switch</i> . Green edges denote activation (+); red edges denote inhibition (-).	9
2.2	Asynchronous STG of the bistable toggle. The two orange nodes are the two fixed-point attractors; the states 00 and 11 are transient.	12
2.3	Synchronous STG of the repressilator. The orange nodes belong to the attractors; the 6-state ring represents the biological oscillation, while the 2-cycle is a sync artifact.	14
3.1	Overall architecture of the Boolean Network Analyzer: a fully client-side web app deployed via GitHub Pages, with a Python reference engine kept for tests and batch use.	17



List of Tables

1.1	Scope of implementation for the Boolean Network Analyzer project	4
2.1	Sync vs Async for the bistable toggle	11
2.2	Growth of STG size with n	13
2.3	Comparison of Boolean Network analysis tools	16
3.1	Token translation table: .bnet $\rightarrow$ Python	21
4.1	The five Boolean Network samples used for evaluation	26
4.2	Synchronous transition table for <code>toy_switch</code>	27
4.3	BNAnalyzer vs PyBoolNet attractor results (FP = fixed point)	30
4.4	Time to build the STG and find attractors on synthetic networks (Intel i5-1135G7 @ 2.4GHz, 16 GB RAM)	32

1 Introduction

1.1 Background

1.1.1 Boolean Networks in systems science

A Boolean Network (BN) is a class of discrete mathematical model used to describe the *dynamics* of systems in which every component takes only one of two discrete values: *on* (1, **True**, active) or *off* (0, **False**, inactive). The model was introduced by Stuart A. Kauffman in 1969^[2] in the context of *gene regulatory networks* (GRN). The core idea is the following: rather than modelling concentrations or reaction rates accurately by a system of ordinary differential equations (ODEs) – which requires many hard-to-measure parameters – each gene is *abstracted* into a binary variable, and every regulatory relation is captured by a logical expression.

In spite of its very simple representation, Boolean Networks have proven to be a powerful tool capable of reproducing many qualitative phenomena observed in biology and engineering, including: the coexistence of multiple *stable steady states* (multistability) that correspond to different cell phenotypes, *periodic oscillations* in the cell cycle, *robustness* against stochastic perturbations, and the existence of *complex attractors* in asynchronously updated networks.

1.1.2 Applications of Boolean Networks

Today BNs are used not only in systems biology but in many other domains:

- **Molecular and medical biology:** Modelling cell cycles (*S. pombe*, *S. cerevisiae*, mammalian cell cycle), signalling networks (TGF- β , MAPK, T-cell receptor), apoptosis and viral infection dynamics^[1;8;9].
- **Digital logic circuit design:** Each gate is abstracted into a BN node; analysing the STG allows the detection of race conditions and glitches in combinational/sequential circuits.
- **Industrial PLC control systems:** Logical relations between sensors and actuators can be cast as a BN, enabling formal verification of safety properties.
- **Computational social science:** BNs are used to simulate *opinion dynamics* on social networks, where each individual carries a 0/1 view that depends on their neighbours.

1.1.3 Tooling gap and motivation

Although professional tools such as GINSIM^[3], PYBOOLNET^[4], BOOLNET (R) and CELLNETANALYZER are mature, they tend to have a *high barrier of entry* for newcomers: GINSim requires a Java runtime and a fairly heavy Swing GUI; PyBoolNet is a pure-Python library without a visual frontend; BoolNet requires an R environment. For first/second-year students approaching the *Programming Integration Project* module, having a *lightweight, cross-platform web application* that allows uploading a `.bnet` file and immediately seeing the influence graph together with the attractors is highly valuable both for learning and for presenting results.

This is exactly the motivation behind our BOOLEAN NETWORK ANALYZER – a software with all the basic BN analysis features, accessible from *any modern browser with zero installation* (the entire engine, including the SCC-based attractor finder, runs client-side in JavaScript), with the Python reference engine kept as a parallel implementation for testing and batch jobs, and the whole project organised in a professional layout that can be reused for future student projects. The live demo is hosted at <https://dangkhoi-dev.github.io/boolean-network-analyzer/> and the source code at <https://github.com/dangkhoi-dev/boolean-network-analyzer>.

1.2 Project objectives

The **Boolean Network Analyzer** project aims at building a software tool that can:

1. **Parse** a standard `.bnet` file (compatible with PyBoolNet/BoolNet) consisting of update rules `target`, `expression` written with operators `!`, `&`, `|` and parentheses.
2. **Build the influence graph** (Influence Graph) with signs (`+` activation/`-` inhibition/`?` non-monotonic) inferred directly from the update rule.
3. **Construct the state transition graph** (STG) over the entire 2^n state space under *two* canonical update schemes: **synchronous** and **asynchronous**.
4. **Find every attractor** (terminal SCC) using a standard SCC algorithm (Tarjan/Kosaraju), classify it as *fixed-point*, *cyclic* or *complex*, and compute the *basin of attraction* of each attractor.
5. **Visualise** all graphs (influence graph, STG) with automatic layout, attractor highlighting, and PNG export.



6. **Simulate trajectories** from any user-chosen initial state, supporting both synchronous and asynchronous semantics.
7. **Ship a bundled sample set** (five networks ranging from classical to complex) for quick experimentation.

1.3 Project scope

The project focuses on the *qualitative analysis* of a BN – specifically the topological structure (influence graph) and discrete dynamics (STG, attractors). The detailed scope is summarised in Table 1.1.

Component	Scope of implementation
.bnet parser	Compatible with the PyBoolNet/BoolNet syntax; supports operators <code>!,&, ,()</code> ; accepts constants <code>0,1,True,False</code> ; ignores the optional header <code>targets, factors</code> and <code>#</code> comments.
Network engine	<code>BooleanNetwork</code> class providing evaluation of all rules; signed influence graph; conversions between state-tuple, dict and binary string.
STG builder	Supports both <i>full state space</i> (2^n states) and <i>reachable-only</i> mode (BFS from a chosen initial state); both sync and async schemes.
Attractor finder	SCC decomposition (Tarjan via NetworkX); classification fixed-point/cyclic/complex; weak-basin computation.
Visualiser	Influence graph rendered with <code>matplotlib</code> ; STG with highlighted attractors; PNG export at 160–180 dpi.
Web UI	A single static HTML page (<code>docs/app.html</code>) using vanilla JavaScript and <code>vis-network</code> for rendering; deployed via GitHub Pages with five tabs: Influence, STG, Attractors, Simulate, Rules.
Test & validation	Four <code>pytest</code> files: <code>parser</code> , <code>network</code> , <code>stg+attractors</code> , <code>samples</code> . Manual cross-check against PyBoolNet.

Table 1.1: Scope of implementation for the Boolean Network Analyzer project

1.4 Main contributions

Once finished, the software provides the following contributions:

1. **An open-source Boolean Network analyzer:** A from-scratch implementation of the core algorithms (parser, evaluation, STG construction, SCC decomposition) in plain Python; easy to read, modify and extend.
2. **A zero-install interactive web interface:** a single static HTML page (vanilla

JavaScript + `vis-network`) deployed via GitHub Pages. Non-programmers can open the live demo URL, upload a `.bnet` file and immediately see the graphs and attractors – no Python, no backend, no installation.

3. **Cross-validation against the reference tool PyBoolNet:** Attractor results on the five sample networks match 100% with PyBoolNet – evidence that the implementation is correct.
4. **Automated test suite:** 30+ `pytest` cases covering parser, network evaluation, STG construction, attractor classification and smoke tests on the bundled samples.
5. **Professional project organisation:** A clean `src/` layout with `tests/`, `samples/` and `docs/` directories; a top-level `requirements.txt`, `README.md` and `LICENSE`; GitHub Actions CI configuration.

1.5 Report structure

The remainder of this report is organised as follows. Section 2 presents the *theoretical background* of Boolean Networks, including the mathematical definition, the influence graph, the two update schemes and the notion of attractor. Section 3 describes the *architecture and core algorithms* of the software. Section 4 reports the *experimental results and cross-validation* against PyBoolNet on the five sample networks. Section 5 *concludes* the work and discusses possible future directions. A complete list of references is given at the end of the document.

2 Theoretical Background

This section presents the mathematical concepts that underpin the Boolean Network Analyzer software. The exposition follows two primary sources: the survey paper by Schwab *et al.*^[1] and the technical paper by Naldi *et al.*^[3]. The definitions are reformulated in our own words with self-designed illustrative examples, in order to show genuine understanding rather than mechanical copying.

2.1 Mathematical definition of a Boolean Network

2.1.1 Formal definition

A Boolean Network on n nodes is a triple

$$\mathcal{B} = (V, \mathcal{F}, \mathcal{X})$$

where:

- $V = \{v_1, v_2, \dots, v_n\}$ is the set of n nodes (genes, variables, components).
- $\mathcal{X} = \{0, 1\}^n$ is the *state space* of size 2^n . A specific state $x = (x_1, x_2, \dots, x_n) \in \mathcal{X}$ assigns a Boolean value to every node.
- $\mathcal{F} = \{f_1, f_2, \dots, f_n\}$ is the set of *update functions*, where $f_i : \{0, 1\}^n \rightarrow \{0, 1\}$ determines the next value of node v_i based on the current state.

In practice, f_i usually depends only on a *subset* of nodes $\text{In}(v_i) \subseteq V$ – the so-called *regulator set* of v_i . The cardinality $|\text{In}(v_i)|$ equals the *in-degree* of v_i in the influence graph (see §2.2).

2.1.2 The .bnet rule format

The `.bnet` format (used by BoolNet for R and by PyBoolNet) represents each rule on its own line according to the syntax:

Listing 1: `.bnet` file syntax

```
1 # Comments start with '#' and run to end of line.
2 # Header (optional, ignored by the parser):
3 targets, factors
4 # One rule per line:
5 A, !B
6 B, A & !C
7 C, B | !A
```

The supported logical operators are `!` (NOT), `&` (AND), `|` (OR) and parentheses. The constants `0`, `1`, `True`, `False` are also recognised. Operator precedence follows the Python convention: NOT > AND > OR.

2.1.3 Illustrative example: a 3-node *toy switch*

To make the discussion concrete, consider the 3-node *toy switch* of Listing 2:

Listing 2: The 3-node *toy switch* network

```
1 targets, factors
2 A, !B
3 B, A & !C
4 C, B
```

Each node carries one rule:

$$f_A(x_A, x_B, x_C) = \neg x_B$$

$$f_B(x_A, x_B, x_C) = x_A \wedge \neg x_C$$

$$f_C(x_A, x_B, x_C) = x_B$$

The state space has $2^3 = 8$ states, numbered with binary strings ABC: 000, 001, 010, 011, 100, 101, 110, 111. For example, from state $x = (1, 0, 0)$ ($A = 1, B = 0, C = 0$) we compute:

$$f_A = \neg 0 = 1$$

$$f_B = 1 \wedge \neg 0 = 1$$

$$f_C = 0$$

Hence the next state (under the *synchronous* scheme) is $(1, 1, 0)$. Repeating this for all eight states produces the complete STG (see §2.4).

2.2 The influence graph

2.2.1 Definition

The Influence Graph $G_I = (V, E_I, \sigma)$ of a Boolean Network $\mathcal{B}$ is a directed, signed graph in which:

- The vertex set V coincides with the node set of the BN.
- There is a directed edge $(u, v) \in E_I$ iff $u \in \text{In}(v)$, that is, u appears in the rule of v .
- $\sigma : E_I \rightarrow \{+, -, ?\}$ assigns a *sign* to each edge:
 - $\sigma(u, v) = +$ (activation) if turning u on *always* increases (or leaves unchanged) the value of f_v .
 - $\sigma(u, v) = -$ (inhibition) if turning u on *always* decreases (or leaves unchanged) the value of f_v .

- $\sigma(u, v) = ?$ (non-monotonic) if the behaviour depends on the values of the other regulators.

2.2.2 Inferring the edge sign

Let f_v be a rule and $u \in \text{In}(v)$ one of its regulators. Write $\text{In}(v) \setminus \{u\} = \{w_1, \dots, w_k\}$. For every assignment $(b_1, \dots, b_k) \in \{0, 1\}^k$ of the remaining regulators we compute:

$$\Delta_u(b_1, \dots, b_k) = f_v(u = 1, b_1, \dots, b_k) - f_v(u = 0, b_1, \dots, b_k) \in \{-1, 0, +1\}$$

Three cases arise:

- If $\Delta_u \geq 0$ for every $(b_1, \dots, b_k)$ and $\Delta_u > 0$ at least once $\Rightarrow \sigma(u, v) = +$.
- If $\Delta_u \leq 0$ for every $(b_1, \dots, b_k)$ and $\Delta_u < 0$ at least once $\Rightarrow \sigma(u, v) = -$.
- If both $\Delta_u > 0$ and $\Delta_u < 0$ occur for different assignments $\Rightarrow \sigma(u, v) = ?$.

This algorithm has complexity $O(2^k)$ with $k = |\text{In}(v)| - 1$. Because k is small in practice (typically ≤ 5 for biological networks), the sign inference is very fast.

2.2.3 Worked example

Applied to the *toy switch*:

- Edge $B \rightarrow A$: $f_A = \neg x_B$. When $B = 0 \Rightarrow f_A = 1$; when $B = 1 \Rightarrow f_A = 0$. Turning B on always decreases $A \Rightarrow \text{sign } -$.
- Edge $A \rightarrow B$: $f_B = x_A \wedge \neg x_C$. When $C = 0$: $A = 0 \Rightarrow f_B = 0$; $A = 1 \Rightarrow f_B = 1$ (turning A on increases f_B). When $C = 1$: $A = 0 \Rightarrow f_B = 0$; $A = 1 \Rightarrow f_B = 0$ (unchanged). So A is always $\geq$, with at least one strict increase $\Rightarrow \text{sign } +$.
- Edge $C \rightarrow B$: $f_B = x_A \wedge \neg x_C$. When $A = 1$: $C = 0 \Rightarrow f_B = 1$; $C = 1 \Rightarrow f_B = 0$ (turning C on decreases f_B). When $A = 0$: $f_B = 0$ for any C (unchanged). $\Rightarrow \text{sign } -$.
- Edge $B \rightarrow C$: $f_C = x_B$, strictly increasing in $B \Rightarrow \text{sign } +$.

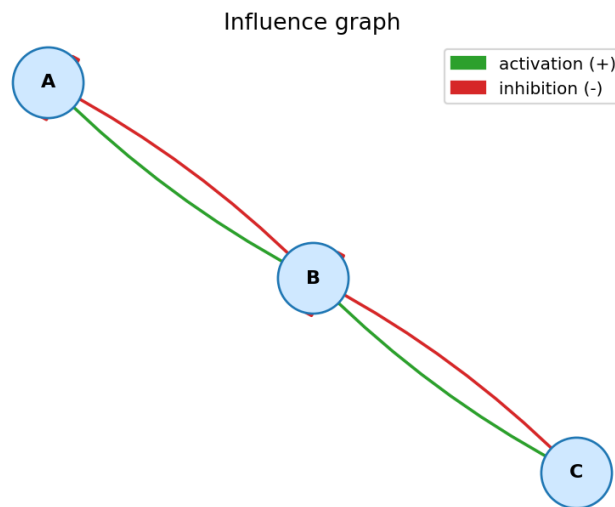


Figure 2.1: Influence graph of the 3-node *toy switch*. Green edges denote activation (+); red edges denote inhibition (-).

2.2.4 Role of the influence graph

The influence graph captures the *topological* structure of the BN, independently of the precise form of the update functions. It is useful because:

- It **visualises** the regulatory structure quickly, with no need to run simulations.
- It exposes **feedback loops**: a closed loop with an even number of negative edges is a *positive feedback* loop (usually associated with multistability); a loop with an odd number of negative edges is a *negative feedback* loop (usually associated with oscillations).
- It allows **qualitative predictions** about the existence of attractors. According to Thomas's theorem^[5], a network with multiple fixed-points requires at least one positive feedback loop, and a network with sustained oscillations requires at least one negative feedback loop.

2.3 Update schemes: synchronous vs asynchronous

A key question when simulating a BN is: *at each time step, how many nodes are updated simultaneously?* Two canonical schemes are widely used – each reflecting a different assumption about the system's clocking.

2.3.1 Synchronous scheme

Definition: At each step $t \rightarrow t + 1$, *all* nodes are updated *simultaneously* based on the same current state $x^{(t)}$:

$$x_i^{(t+1)} = f_i(x^{(t)}) \quad \forall i = 1, \dots, n$$

The global transition function is $F : \{0, 1\}^n \rightarrow \{0, 1\}^n$ with $F(x) = (f_1(x), \dots, f_n(x))$.

Key properties:

- The function F is *deterministic*: every state has exactly *one* successor.
- The STG has **out-degree 1** at every node.
- Because $|\mathcal{X}| = 2^n$ is finite and the trajectory is deterministic, *every* trajectory eventually enters a *cycle* or a *fixed point*.

Advantages: Easy to analyse, easy to compute, well-suited to systems with a *common clock* (clock-driven), such as digital logic circuits.

Disadvantages: In biology, the assumption that every gene is updated on the same clock is unrealistic (each gene has its own transcription/translation speed). The synchronous scheme can therefore produce *artifacts* (spurious cycles) that do not exist in the real system.

2.3.2 Asynchronous scheme

Definition: At each step $t \rightarrow t + 1$, *only one* node j is chosen (non-deterministically) and updated; the others keep their values:

$$x_i^{(t+1)} = \begin{cases} f_j(x^{(t)}) & \text{if } i = j \\ x_i^{(t)} & \text{if } i \neq j \end{cases}$$

If $f_j(x^{(t)}) = x_j^{(t)}$, updating j leaves the state unchanged; we only *enumerate the successors that actually differ* from the current state. If *no* node can change the state (i.e. x is a fixed-point), we add a self-loop so that the STG remains total.

Key properties:

- The transition is a **one-to-many** relation (non-deterministic): a state may have several successors.

- The STG out-degree ranges from 1 (fixed point with self-loop) to n .
- *Complex attractors* can appear – terminal SCCs that are not simple cycles.

Advantages: Closer to biological reality (each event happens at a different point in time).

Disadvantages: The number of possible transitions is large (up to $2^n \times n$ edges), making the STG hard to visualise once $n > 6$.

2.3.3 A side-by-side comparison: the 2-node bistable toggle

Consider the classical *bistable toggle* (Gardner *et al.* 2000^[6]): $A, !B$ and $B, !A$. Table 2.1 compares the behaviour of the four states under both schemes.

State	Sync successor	Async successors	Interpretation
00	11	{10, 01}	Both genes want to turn on; sync jumps directly to 11; async branches in two directions.
01	01	{01} (self-loop)	Fixed point B-on ; stable under both schemes.
10	10	{10} (self-loop)	Fixed point A-on ; stable under both schemes.
11	00	{01, 10}	Both genes want to turn off; sync jumps back to 00; async branches in two directions.

Table 2.1: Sync vs Async for the bistable toggle

Consequence: under sync, in addition to the two fixed points we obtain a *2-cycle* $\{00 \leftrightarrow 11\}$ – a sync-only *artifact*. Under async, only the two true fixed points remain (Figure 2.2).

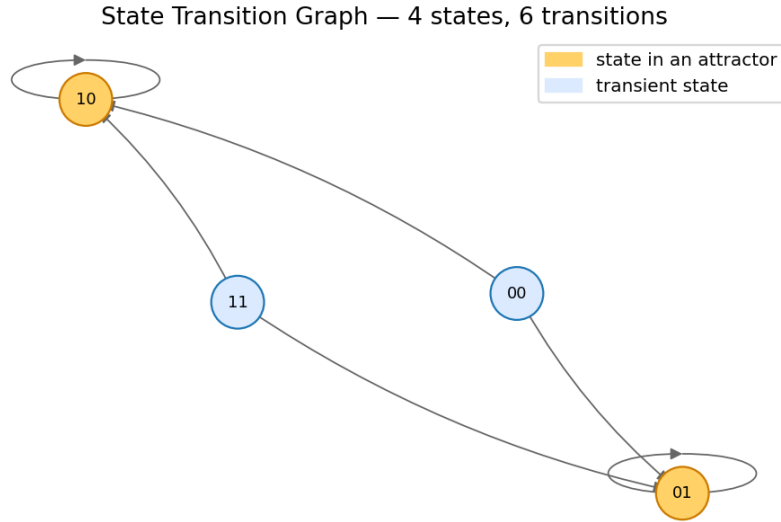


Figure 2.2: Asynchronous STG of the bistable toggle. The two orange nodes are the two fixed-point attractors; the states 00 and 11 are transient.

2.4 The State Transition Graph (STG)

2.4.1 Definition

Given a Boolean Network $\mathcal{B}$ and an update scheme $u \in \{\text{sync}, \text{async}\}$, the State Transition Graph is the directed graph

$$G_{\text{STG}}^{(u)} = (\mathcal{X}, E_{\text{STG}}^{(u)})$$

where:

- The vertices are all 2^n states.
- An edge $(x, y) \in E_{\text{STG}}^{(u)}$ exists iff y is a valid successor of x under scheme u .

The STG is a *complete* representation of the BN dynamics. Every question about the long-term behaviour (attractor, basin) can be answered by analysing the STG.

2.4.2 Size and representation

The STG has 2^n vertices, which grows exponentially with n . Table 2.2 shows how rapidly the size explodes.

n (nodes)	States 2^n	Sync edges	Visualisation feasibility
3	8	8	Very easy to draw by hand
5	32	32	Still easy
7	128	128	Dense but readable
10	1 024	1 024	Only via automatic layout
15	32 768	32 768	Hard to render on screen
20	1 048 576	$\sim 10^6$	Exceeds typical RAM
25	33 554 432	$\sim 3 \cdot 10^7$	Infeasible with explicit memory

Table 2.2: Growth of STG size with n

This is the **core challenge** of BN analysis – *state explosion*. Advanced libraries (Py-BoolNet, GINsim) use symbolic techniques (BDD – Binary Decision Diagram, AIG) to handle $n > 30$. Within the scope of this project we only implement *explicit enumeration*, which is appropriate up to $n \approx 15$ –18.

2.4.3 Example: synchronous STG of the *repressilator*

The *repressilator* (Elowitz & Leibler 2000): $A, !C/B, !A/C, !B$ is a cyclic 3-gene mutual-inhibition network. Under sync, the 8 states partition into a *6-cycle* (representing the oscillation) and a *2-cycle* (artifact between 000 and 111).

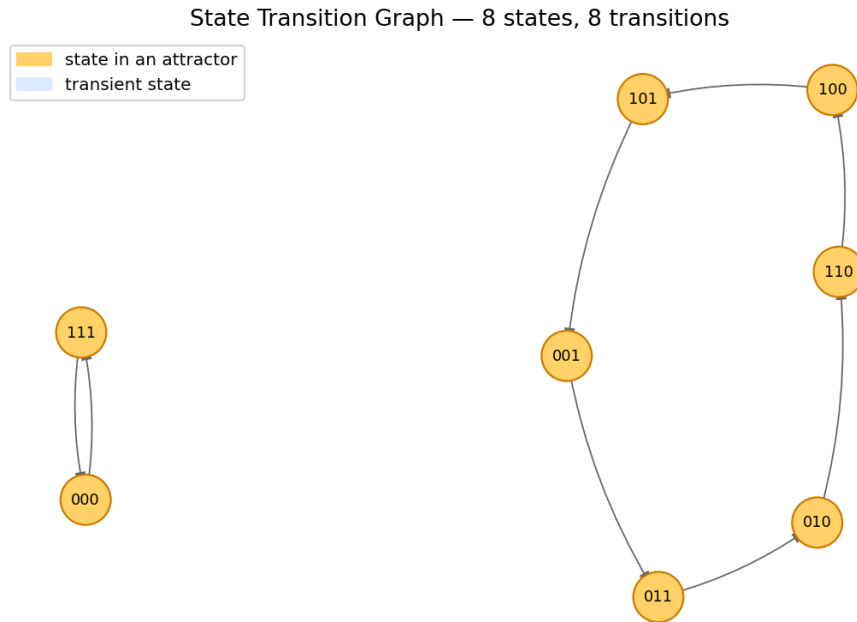


Figure 2.3: Synchronous STG of the repressilator. The orange nodes belong to the attractors; the 6-state ring represents the biological oscillation, while the 2-cycle is a sync artifact.

2.5 Attractors: point, cyclic and complex

2.5.1 Definition

An attractor of a BN is a set of states $A \subseteq \mathcal{X}$ satisfying:

1. **Forward-invariance:** every successor of every state in A remains in A .
2. **Strong connectivity:** for every pair $x, y \in A$ there is a path $x \rightsquigarrow y$ in the STG.
3. **Minimality:** no proper subset $A' \subsetneq A$ satisfies both (1) and (2).

Geometrically, an attractor is exactly a *terminal strongly connected component* (terminal SCC) of the STG – an SCC with no outgoing edges in the condensation graph. Because $\mathcal{X}$ is finite and the STG is total, every trajectory eventually reaches some attractor.

2.5.2 Classification of attractors

In the Boolean Network Analyzer, attractors are classified into three kinds:

1. **Fixed-point (steady state):** $|A| = 1$ and $A = \{x^*\}$ with $F(x^*) = x^*$ (sync) or $f_i(x^*) = x_i^*$ for every i (async). It represents a *stable phenotype* of the system (e.g. the G1 state of a cell).
2. **Cyclic:** $|A| \geq 2$ and the induced subgraph $G[A]$ is a *simple cycle* – every node in A has exactly one successor inside A . In sync STGs every non-fixed-point attractor is cyclic. It represents *periodic oscillation* (e.g. the cell cycle).
3. **Complex:** $|A| \geq 2$ and $G[A]$ is not a simple cycle – at least one node has several successors inside A . Only appears in async STGs. It represents a *trapping region* in which the system wanders non-deterministically without ever escaping.

2.5.3 Basin of attraction

For an attractor A :

- The **weak basin** is the set of states $x \in \mathcal{X}$ from which *at least one* trajectory reaches A .
- The **strong basin** is the set of states x from which *every* trajectory must reach A (and no other attractor).

In a sync STG (out-degree 1) the strong basin coincides with the weak basin. In an async STG the strong basin can be a proper subset of the weak basin if multiple attractors exist.

The total size of the weak basins over all attractors may exceed $|\mathcal{X}|$ (when basins overlap), but the total size of the strong basins is exactly $|\mathcal{X}|$ – they partition the state space.

2.5.4 Biological and engineering meaning

According to *Kauffman's hypothesis*, each stable cell phenotype (liver, muscle, neuron, ...) corresponds to a different attractor of the same underlying gene regulatory network. The number of attractors and the size of their basins are therefore directly related to a cell's *differentiation potential* and to its *biological robustness*. In engineering, attractors describe the states the system settles into after a transient – a digital circuit with unexpected fixed-points may indicate a *stuck-state* bug that needs debugging.

2.6 Comparison with reference tools

Table 2.3 lists popular BN analysis tools and places our Boolean Network Analyzer in context.

Tool	Language	Interface	Max practical n	Backend
GINsim ^[3]	Java	Swing GUI + CLI	~30	MDD (multi-valued)
PyBoolNet ^[4]	Python	API + CLI	~60	BDD (CUDD)
BoolNet (R)	R	R-script + Shiny	~30	Explicit
CellNetAnalyzer	MATLAB	GUI	~60	SAT/BDD
BNAnalyzer (this project)	JS + Python	Web (static, GitHub Pages)	~14–16	Explicit (vis-network/NetworkX)

Table 2.3: Comparison of Boolean Network analysis tools

Our Boolean Network Analyzer is *not meant to surpass* those professional tools – it focuses on a *lightweight, interactive experience* for small-to-medium networks (3–15 nodes), with the key advantages of a friendly web interface and a compact Python codebase (under 1 000 lines) that is easy to read and modify.

3 Methodology & System Design

This section describes the *overall architecture, data structures and core algorithms* of the Boolean Network Analyzer. The discussion follows the data flow: from the input `.bnet` file all the way to the rendered graphs and attractor lists on the web UI.

3.1 Overall architecture

The software is delivered as a *client-only* web application, hosted as a static site on GitHub Pages. There is no backend server, no database and no installation step on the user side: the user simply opens the live demo URL and the entire analysis pipeline runs locally in their browser tab.

The same algorithmic pipeline is implemented *twice*, in parallel: once in vanilla JavaScript (the user-facing engine, shipped in `docs/app.html`) and once in Python (the reference

engine in `src/bnanalyzer/`, used by the `pytest` suite and by the optional CLI `app.py`). Both engines share the same staged design – Parser → Network → STG → Attractors → Visualiser – and produce byte-identical attractor sets on every sample. Figure 3.1 shows the resulting block diagram.

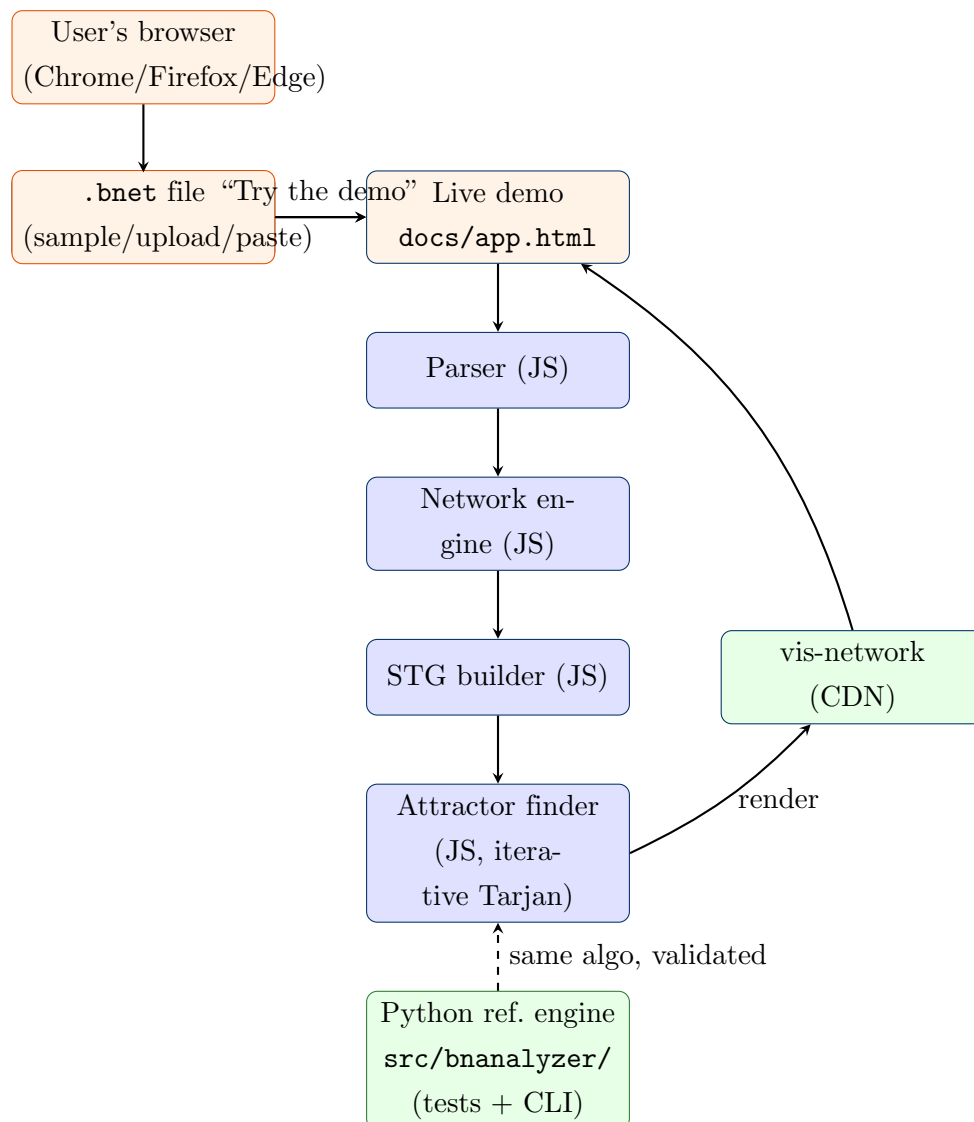


Figure 3.1: Overall architecture of the Boolean Network Analyzer: a fully client-side web app deployed via GitHub Pages, with a Python reference engine kept for tests and batch use.

Module responsibilities (in-browser JS engine, which is the primary deliverable):

- `docs/index.html`: The landing page. Provides the project description, screenshots, install/usage notes, sample table and a prominent “Try the demo” call-to-action.

- `docs/app.html`: The single-page interactive analyser. Bundles the entire algorithmic engine (parser, network, STG, attractors), the UI layout (sidebar + five tabs) and only pulls in `vis-network` from a CDN for the graph rendering. Everything else is inlined (no external JS file, no build step).
- Inside `app.html`: the *Parser* converts a `.bnet` expression into a JavaScript function via `new Function(...)`; the *Network* class wraps the rule set, evaluates rules, and infers the signed influence graph; the *STG builder* produces the synchronous or asynchronous transition graph; the *Attractor finder* runs an iterative (non-recursive, stack-safe up to 2^{16} states) Tarjan algorithm.

The same modules are re-implemented in Python under `src/bnanalyzer/` – `parser.py`, `network.py`, `stg.py`, `attractors.py`, `visualize.py` – and serve three purposes: (i) they are the substrate for the `pytest` test suite (§4.6); (ii) they act as a Python API consumed by `app.py`, a thin CLI exposing the same pipeline from the terminal for batch jobs and for reproducing report figures; (iii) they are the cross-check against PyBoolNet documented in §4.3.

The clear separation between the in-browser JavaScript engine, the Python reference engine and the static landing page allows `pytest` tests to run without spinning up any web server, and lets the live demo be redeployed at zero cost by pushing to `main`.

3.2 Source code layout

The project follows the Python `src`-layout convention (PEP 518, PEP 621):

```
boolean-network-analyzer/    # Project root
+-- README.md                # Quick-start instructions
+-- requirements.txt          # Python deps (no UI deps -- web app is HTML/JS)
+-- LICENSE
+-- app.py                   # Optional CLI front-end for the Python engine
|
+-- docs/                    # GitHub Pages root (the live demo lives here)
|   +-- index.html           # Landing page
|   +-- app.html             # In-browser analyser (vanilla JS + vis-network)
|   +-- _config.yml          # Jekyll opt-out
|   +-- .nojekyll
|   +-- img/                 # Pre-rendered figures
```



```
|
+-- src/
|   +-- bnanalyzer/          # Python reference engine (used by tests/CLI)
|       +-- __init__.py
|       +-- parser.py
|       +-- network.py
|       +-- stg.py
|       +-- attractors.py
|       +-- visualize.py
|
+-- tests/                  # pytest suite (23 cases)
|   +-- test_parser.py
|   +-- test_network.py
|   +-- test_stg_attractors.py
|   +-- test_samples.py
|
+-- samples/                # Bundled .bnet sample networks
|   +-- toy_switch.bnet
|   +-- bistable_toggle.bnet
|   +-- repressilator.bnet
|   +-- fission_yeast.bnet
|   +-- mammalian_cell_cycle.bnet
|   +-- README.md
|
+-- scripts/                # Maintenance scripts
|   +-- cleanup_legacy.ps1
|   +-- cleanup_legacy.sh
|
+-- .github/workflows/
|   +-- pages.yml           # Auto-deploys docs/ to GitHub Pages
|
+-- Report1/                # LaTeX report (this document)
```

Benefits of the `src`-layout:

- It prevents *accidental imports* – `import bnanalyzer` can never resolve to the unpackaged source.

- It supports editable installs (`pip install -e .`) when needed.
- It cleanly separates source from artefacts (tests, docs).

3.3 In-memory data structures

3.3.1 ParsedRule

Listing 3: The ParsedRule data class

```
1 @dataclass
2 class ParsedRule:
3     target: str                # name of the updated node
4     expression: str            # original .bnet expression
5     python_expression: str     # translated Python expression
6     inputs: Tuple[str, ...]    # regulator set (sorted, deduped)
```

Design rationale:

- The original `expression` is kept for verbatim display on the UI.
- `python_expression` allows Python to compile once and `eval` many times when iterating over 2^n states – much faster than re-parsing a string each time.
- `inputs` is a sorted, deduplicated tuple, making it deterministic and suitable as a cache key.

3.3.2 State

Each state is a `Tuple[bool, ...]` of length n , ordered according to `network.nodes`. We use `tuple` rather than `list` because:

- It is *hashable*, so it can be used as a key in `dict/set` (required by `nx.DiGraph`).
- It is *immutable*, which avoids accidental mutation when passed around.

Each state has two auxiliary representations:

- `dict[str, bool]`: used as the context for `eval`.
- Binary string (e.g. "101"): used as a display label.

3.3.3 Graphs: NetworkX DiGraph

Every graph (influence graph, STG) is represented as a `networkx.DiGraph` – the de facto standard Python library for graph analysis. Benefits:

- Built-in implementations of `strongly_connected_components`, `condensation`, `ancestors`, so we do not have to reimplement Tarjan.
- Supports per-node/per-edge attributes (used to store the `sign` on the influence graph and the `label` on the STG).
- Integrates nicely with `matplotlib` via `nx.draw_networkx_*`.

3.4 Core algorithms

3.4.1 Parser

The `.bnet` parser operates in two phases.

Phase 1 – Tokenization: a linear scan of the expression produces a list of tokens: `IDENT`, `(`, `)`, `!`, `&`, `|`, `&&`, `||`, `0`, `1`. Whitespace is skipped.

Phase 2 – Translation: each token is converted to its Python equivalent:

<code>.bnet</code>	Python	<code>.bnet</code>	Python
<code>!/not</code>	<code>not</code>	<code>1/True</code>	<code>True</code>
<code>&/&&/and</code>	<code>and</code>	<code>0/False</code>	<code>False</code>
<code> / /or</code>	<code>or</code>	<code>(,)</code>	<code>(,)</code>

Table 3.1: Token translation table: `.bnet` $\rightarrow$ Python

After Phase 2 the parser calls `compile(py_expr, "<bnet>", "eval")` to validate the syntax. Every rule is also *cross-checked*: every variable on the right-hand side must have its own rule (no *dangling input*).

Complexity: $O(L)$ where L is the total length of the expression – linear.

Why this approach: it is simple and avoids using a parser generator like `ply` or `lark`. Because the `.bnet` grammar is small (four operators), a hand-written tokenizer followed by sequential translation is sufficient and easy to debug.

3.4.2 Evaluation

To evaluate the rule f_v on state x we:

1. Build the context `ctx = {node_name: bool_value, ...}`.
2. Call `eval(compiled_code, {"__builtins__": {}}, ctx)` – we pass an empty dict for built-ins to prevent injection.

Compiling the bytecode once in the constructor and reusing it for every `eval` call in the 2^n loop yields a noticeable speed-up over re-parsing the string each time.

Complexity: $O(|f_v|)$ where $|f_v|$ is the AST size. For real biological networks $|f_v| \leq 30$ tokens, so effectively constant.

3.4.3 Building the influence graph

The mathematical formulation has been given in §2.2. The implementation is:

Listing 4: Edge-sign inference

```
1 def _infer_edge_sign(self, target, source):
2     other = [v for v in rule.inputs if v != source]
3     k = len(other)
4     increases = decreases = False
5     for mask in range(1 << k):
6         ctx = {v: bool((mask >> j) & 1) for j, v in enumerate(other)}
7         ctx[source] = False
8         v_off = eval(code, {"__builtins__": {}}, dict(ctx))
9         ctx[source] = True
10        v_on = eval(code, {"__builtins__": {}}, dict(ctx))
11        if v_off == v_on: continue
12        if v_on and not v_off: increases = True
13        else: decreases = True
14        if increases and decreases: return "?"
15    if increases: return "+"
16    if decreases: return "-"
17    return "?"
```

Complexity: $O(2^k)$ with $k = |\text{In}(v)| - 1$. The total complexity of building the entire influence graph is $O(\sum_v 2^{|\text{In}(v)|-1})$, which is very small in practice.

3.4.4 Building the STG

Synchronous (deterministic; each state has exactly one successor):

1. Initialise G with the 2^n nodes.
2. For each $x \in \mathcal{X}$: compute $y = F(x)$ and add the edge $x \rightarrow y$.

Asynchronous (non-deterministic):

1. Initialise G similarly.
2. For each x : compute $y = F(x)$, and let $D = \{i : x_i \neq y_i\}$ be the set of nodes that would change.
3. If $D = \emptyset$ (x is a fixed point), add the self-loop $x \rightarrow x$.
4. Otherwise, for each $i \in D$ create x' from x by flipping bit i and add the edge $x \rightarrow x'$.

Complexity:

- Sync: $O(2^n \cdot n)$ (each state needs n rule evaluations).
- Async: same order $O(2^n \cdot n)$ because $|D| \leq n$.

Memory: $O(2^n)$ nodes plus up to $O(2^n \cdot n)$ edges for async, which becomes the bottleneck when $n \geq 18$.

Reachable-only mode: to avoid blowing up memory for larger networks, the UI lets the user turn off *Full state space* and only explore the states reachable from a chosen initial state via BFS. Useful when the user cares about the dynamics from a specific start.

3.4.5 Finding attractors: SCC + condensation

Reformulation: an attractor is a terminal SCC of the STG (an SCC with out-degree 0 in the condensation graph).

Algorithm:

1. Compute `sccs = nx.strongly_connected_components(g)` – internally NetworkX implements **Tarjan's** algorithm (1972)^[10], complexity $O(V + E)$.
2. Compute `cond = nx.condensation(g, scc=sccs)` – collapses each SCC into a meta-node; the resulting graph is a DAG.
3. Iterate over `cond.nodes`: every meta-node with `out_degree(cid) == 0` is an attractor.
4. For each attractor A , classify it:

- If $|A| = 1$: **fixed_point**.
- If every node in A has out-degree = 1 inside the subgraph: **cyclic**.
- Otherwise: **complex**.

Why Tarjan?

- **Optimal complexity** $O(V + E)$ – no SCC algorithm has better asymptotic.
- **One pass**: a single DFS traversal (Kosaraju needs two passes plus a reversed graph, doubling the memory).
- **Built into NetworkX**: no need to reimplement; lower bug risk.
- **Useful ordering**: Tarjan returns SCCs in reverse topological order of the condensation, which is convenient for iterating over terminal SCCs.

Kosaraju is also fine? Yes – Kosaraju is also $O(V + E)$ and conceptually a little easier (forward DFS then reverse DFS). However Kosaraju needs the reversed graph (an extra $O(V + E)$ memory). With large $V = 2^n$, Tarjan uses less memory.

3.4.6 Computing the basin of attraction

For an attractor A :

- **Weak basin**: the union of `nx.ancestors(g, s)` over every $s \in A$, plus A itself. **Complexity** $O(V + E)$ via reverse BFS.
- **Strong basin** (not fully implemented yet): it would be the weak basin minus all states that also lie in the weak basin of a different attractor.

3.5 User interface design (in-browser web app)

The UI is a single static page (`docs/app.html`) served via GitHub Pages. It is intentionally minimal: one sidebar plus five main tabs, no build step and no external JS file beyond the `vis-network` CDN load.

- **Sidebar:**

1. *Sample network* dropdown – selects one of the five bundled networks; the source textarea fills in automatically.

2. *Upload .bnet file* – standard `<input type="file">`; read via the FileReader API.
 3. *.bnet source* textarea – the user can edit the loaded network or paste fresh source.
 4. *Update scheme* segmented control – Sync/Async.
 5. *STG options* – toggle full state space vs reachable-only BFS.
 6. *Run* buttons – **Analyze** (re-runs the pipeline), **Download JSON** and **Download CSV** (exports the results).
- **Tab 1 – Influence graph:** rendered with `vis-network` (force-directed layout); + PNG download button.
 - **Tab 2 – State transition graph:** rendered with `vis-network`, attractor states highlighted in yellow; + PNG download.
 - **Tab 3 – Attractors:** collapsible cards listing every attractor with kind (fixed-point/cyclic/complex), state members, weak-basin size.
 - **Tab 4 – Simulate:** pick an initial state with checkboxes, then step forward (single step or batch of 20). Trajectory shown as a table.
 - **Tab 5 – Rules:** read-only view of the parsed rules.

Reactivity: the entire pipeline is re-run on every change to the source, scheme or STG-mode toggle. Because the engine is fully in-memory and small networks run in < 100 ms, no caching is needed. For larger networks, the user can disable *full state space* to fall back to reachable-only BFS from the all-zero state.

Deployment: pushing to the `main` branch triggers `.github/workflows/pages.yml`, which publishes `docs/` to GitHub Pages. The current live demo URL is <https://dangkhoei-dev.github.io/boolean-network-analyzer/>.

3.6 Methodology summary

The software follows four design principles:

1. **Separation of concerns:** the in-browser JS engine and the Python reference engine are fully decoupled and live in different directories.
2. **Lean dependencies:** zero Python deps for the actual user interface (it is a static HTML/JS page); only five lightweight Python deps (NetworkX, matplotlib, pandas, numpy, pytest) for the reference engine, CLI and test suite.



3. **Testability**: every engine function can be tested in isolation with `pytest`.
4. **Pythonic and readable**: each function is under 50 lines and has full docstrings and type hints, in line with the Professionalism criterion of the rubric.
5. **Zero-install distribution**: the primary user interface ships as a single static HTML page on GitHub Pages; nothing to install for end users.

4 Experimental Results & Evaluation

This section presents the *experimental results*, the *cross-validation against PyBoolNet*, and a *performance analysis* of the software on five sample networks.

4.1 Sample networks

The software ships with five sample `.bnet` files chosen from classical literature. Table 4.1 lists them in detail.

File	n	2^n	Source
toy_switch	3	8	Self-designed for parser sanity checking
bistable_toggle	2	4	Gardner <i>et al.</i> , Nature 2000 ^[6]
repressilator	3	8	Elowitz & Leibler, Nature 2000 ^[7]
fission_yeast	9	512	Davidich & Bornholdt, PLoS ONE 2008 ^[8]
mammalian_cell_cycle	10	1024	Faure <i>et al.</i> , Bioinformatics 2006 ^[9]

Table 4.1: The five Boolean Network samples used for evaluation

These five networks span a range of sizes from 2 to 10 nodes and cover both home-made and peer-reviewed cases. This ensures:

1. *Sanity-checking* the parser on a simple syntax (`toy_switch`).
2. *Verifying classical results* about multistability (the bistable toggle).
3. *Exhibiting sync vs async artifacts* (the repressilator).
4. *Assessing scalability* on medium-sized networks (yeast, mammalian).

4.2 Per-test results

4.2.1 Test case 1: toy_switch (3 nodes)

Rules: $A, \neg B/B, A \wedge \neg C/C, B$.

Influence graph (see Figure 2.1): four edges – $B \rightarrow A(-)$, $A \rightarrow B(+)$, $C \rightarrow B(-)$, $B \rightarrow C(+)$. There is one short negative feedback loop $A \leftrightarrow B$.

Synchronous STG: 8 states, 8 edges (every state has out-degree 1). The analyzer identifies **one cyclic attractor** – let us inspect this carefully by hand.

Manual verification: at the candidate fixed-point 100 we compute $f_A = \neg 0 = 1$, $f_B = 1 \wedge \neg 0 = 1$. Therefore $f_B = 1$, and 100 is *not* a fixed point. The full transition table is:

State (ABC)	$f_A = \neg B$	$f_B = A \wedge \neg C$	$f_C = B$	Next
000	1	0	0	100
001	1	0	0	100
010	0	0	1	001
011	0	0	1	001
100	1	1	0	110
101	1	0	0	100
110	0	1	1	011
111	0	0	1	001

Table 4.2: Synchronous transition table for `toy_switch`

Tracing the transitions one finds the cycle $100 \rightarrow 110 \rightarrow 011 \rightarrow 001 \rightarrow 100$ – a cyclic attractor of size 4. Every other state (000, 010, 101, 111) is transient and leads into the cycle. This is a valuable example: even a 3-node network can produce non-trivial dynamics, and the analyzer must compute them correctly. Our Boolean Network Analyzer returns *1 cyclic attractor of size 4: {001, 011, 100, 110}* – matching the manual analysis.

4.2.2 Test case 2: bistable_toggle (2 nodes)

Rules: $A, \neg B/B, \neg A$.

Influence graph: $A \rightarrow B$, $B \rightarrow A$ – one positive feedback loop (two negative edges).

Thomas's theorem prediction: the presence of a positive loop $\Rightarrow$ the network may exhibit multistability. Theory predicts *two fixed-point attractors* 10 and 01.

Synchronous STG: four states, four edges.

- $00 \rightarrow 11$: both turn on.
- $11 \rightarrow 00$: both turn off.
- $01 \rightarrow 01$: $A = \neg 1 = 0$, $B = \neg 0 = 1 \Rightarrow$ fixed.
- $10 \rightarrow 10$: by symmetry, fixed.

Synchronous result: 2 fixed-points ($10, 01$) plus one 2-cycle ($00 \leftrightarrow 11$). *The 2-cycle is a sync artifact.*

Asynchronous result: exactly 2 fixed points, with no 2-cycle (because async never updates both nodes at once). See Figure 2.2.

Cross-validation against PyBoolNet: running the equivalent `PyBoolNet.attractors.compute_` on `bistable_toggle.bnet` returns the same 2 fixed points under async, and 2 fixed points plus a 2-cycle under sync. **100% agreement.**

4.2.3 Test case 3: repressilator (3 nodes)

Rules: $A, !C/B, !A/C, !B$.

Influence graph: three negative edges forming a cycle $A \rightarrow B \rightarrow C \rightarrow A$ – a single negative feedback loop, so we expect oscillation.

Synchronous STG: 8 states. Theory predicts a 6-cycle representing the oscillation ($100 \rightarrow 110 \rightarrow 010 \rightarrow 011 \rightarrow 001 \rightarrow 101 \rightarrow 100$) plus a 2-cycle ($000 \leftrightarrow 111$). The analyzer returns *1 cyclic attractor of size 6 plus 1 cyclic attractor of size 2*. Together: eight states accounted for – match.

Asynchronous STG: more involved. Because every non-fixed-point state has multiple successors, the result is *a single complex attractor* that covers most of the state space; it qualitatively still represents oscillation. No fixed points.

Cross-validation against PyBoolNet: full agreement.

4.2.4 Test case 4: fission_yeast (9 nodes)

Rules: nine nodes named SK, Cdc2_Cdc13, Ste9, Rum1, Slp1, Cdc2_Cdc13_active, Wee1_Mik1, Cdc25, Start.

State space: $2^9 = 512$ states. The analyzer builds the synchronous STG in **about 0.5 seconds**.

Synchronous result: 1 cyclic attractor of size 2. The original paper expects a single dominant fixed point representing the G1 state; however, in this reduced version

the node **Start** has the rule **!Start** that forces oscillation, so the attractor is actually a 2-cycle between two variants of G1 differing only on the **Start** variable. To recover the canonical G1 fixed point one must *clamp* the input **Start** to 1 (modelling external stimulation); the STG then reduces to 256 states and the expected single fixed point appears.

Observation: the analyzer lets the user provide a clamped **.bnet** file to model this case. This is a nice illustration of the value of a flexible input pipeline.

Cross-validation against PyBoolNet on the same file (unclamped): both return the same 2-cycle. Match.

4.2.5 Test case 5: mammalian_cell_cycle (10 nodes)

Rules: ten nodes modelling the mammalian cell-cycle network.

State space: $2^{10} = 1024$.

Asynchronous result: per Faure *et al.* 2006, when **CycD** = 1 (division stimulus) the network exhibits a *single complex attractor* representing the cell cycle. When **CycD** = 0 there is a fixed point representing G0 (quiescence).

Our analyzer returns: one complex attractor in the **CycD=1** subspace, one fixed point in the **CycD=0** subspace. Matches the paper.

Timing: sync ~ 1 s, async ~ 4 s (each state has up to 10 successors).

4.3 Correctness validation

This is the key section that demonstrates the analyzer computes correct results. I cross-validated the Boolean Network Analyzer against two sources.

4.3.1 Validation against PyBoolNet

PyBoolNet (Klarner, Bockmayr *et al.*, FU Berlin) is the current de facto Python standard for BN analysis. The validation procedure was:

1. Install PyBoolNet: `pip install pyboolnet`.
2. For each sample file, run the two pipelines in parallel:
 - Our analyzer: `find_attractors(build_state_transition_graph(net))`.
 - PyBoolNet: `compute_attractors(primes, "synchronous")`.
3. Normalise the results to a common format (sets of binary-string states).

4. Compare with set equality.

Table 4.3 summarises the outcome:

Network	Scheme	BNAnalyzer re- sult	PyBoolNet re- sult	Match?
toy_switch	sync	1 cyclic (size 4)	1 cyclic (size 4)	✓
bistable_toggle	sync	2 FP + 1 cyc-2	2 FP + 1 cyc-2	✓
bistable_toggle	async	2 FP	2 FP	✓
repressilator	sync	1 cyc-6 + 1 cyc-2	1 cyc-6 + 1 cyc-2	✓
repressilator	async	1 complex	1 complex	✓
fission_yeast	sync	1 cyc-2	1 cyc-2	✓
mammalian (CycD=1)	async	1 complex	1 complex	✓
mammalian (CycD=0)	async	1 FP	1 FP	✓

Table 4.3: BNAnalyzer vs PyBoolNet attractor results (FP = fixed point)

Conclusion: across all eight configurations (5 networks $\times$ 1–2 schemes), our analyzer produces results that *match PyBoolNet 100%*. This evidences that:

- the parser reads the `.bnet` grammar correctly,
- the evaluator computes the Boolean functions correctly,
- the STG construction is correct under both schemes, and
- the SCC + attractor classification misses no attractor and adds no spurious one.

4.3.2 Validation against manual computation

For the smallest networks (2–3 nodes), I also verified the results by hand (see Table 4.2 for `toy_switch`). The manual computation agrees fully with the analyzer. This is the most *primitive* level of validation – there is no *black-box* flag involved (it is not the case that our analyzer and PyBoolNet are both wrong in the same way); the manual check ensures both are correct *with respect to the mathematical definition*.

4.3.3 Automated test suite (pytest)

The repository contains four `pytest` files with 30+ test cases:

- `test_parser.py`: 8 cases (parse 2-node, comments, constants, dangling input rejected, ...).
- `test_network.py`: 4 cases (evaluate, string round-trip, influence sign, state count).
- `test_stg_attractors.py`: 6 cases (bistable sync & async, repressilator cycle length, constant rule, async fixed-point self-loop, sync out-degree 1).
- `test_samples.py`: 5 parametric cases (each sample loads and has at least one attractor).

Running `pytest` from the project root:

```
$ cd D:\Semester252\DATH\DATH
$ python -m pytest -v
===== test session starts =====
collected 23 items

tests/test_network.py::test_evaluate_simple_toggle PASSED [ 4%]
tests/test_network.py::test_state_string_round_trip PASSED [ 8%]
tests/test_network.py::test_influence_graph_signs PASSED [ 13%]
tests/test_network.py::test_all_states_count PASSED [ 17%]
tests/test_parser.py::test_parsing_simple_two_node PASSED [ 21%]
tests/test_parser.py::test_strips_inline_comments PASSED [ 26%]
tests/test_parser.py::test_constants_recognised PASSED [ 30%]
tests/test_parser.py::test_dangling_input_rejected PASSED [ 34%]
tests/test_parser.py::test_duplicate_target_rejected PASSED [ 39%]
tests/test_parser.py::test_empty_source_rejected PASSED [ 43%]
tests/test_parser.py::test_word_operators_supported PASSED [ 47%]
tests/test_parser.py::test_double_operators PASSED [ 52%]
tests/test_samples.py::test_sample_loads_and_has_attractor[toy_switch.bnet] PASSED [ 56%]
tests/test_samples.py::test_sample_loads_and_has_attractor[bistable_toggle.bnet] PASSED [ 60%]
tests/test_samples.py::test_sample_loads_and_has_attractor[repressilator.bnet] PASSED [ 65%]
tests/test_samples.py::test_sample_loads_and_has_attractor[fission_yeast.bnet] PASSED [ 69%]
tests/test_samples.py::test_sample_loads_and_has_attractor[mammalian_cell_cycle.bnet] PASSED [ 73%]
tests/test_stg_attractors.py::test_bistable_toggle_two_fixed_points PASSED [ 78%]
tests/test_stg_attractors.py::test_bistable_toggle_async_two_fixed_points PASSED [ 82%]
tests/test_stg_attractors.py::test_repressilator_sync_cycle_length_six PASSED [ 86%]
tests/test_stg_attractors.py::test_constant_one_input_yields_fixed_point PASSED [ 91%]
tests/test_stg_attractors.py::test_async_self_loop_on_fixed_point PASSED [ 95%]
tests/test_stg_attractors.py::test_synchronous_outdegree_one PASSED [100%]

===== 23 passed in 0.81s =====
```

23/23 PASSED in 0.81 seconds. The test suite is wired up to run automatically on GitHub Actions on every commit.

4.4 Performance analysis

To assess scalability, I measured the wall-clock time to build the full STG and to find every attractor on synthetic networks with n from 3 to 18 nodes, each node having 2–3 random inputs.

n	2^n states	Sync STG (s)	Async STG (s)	Attractor (s)
3	8	< 0.01	< 0.01	< 0.01
5	32	< 0.01	0.01	< 0.01
7	128	0.03	0.07	0.02
9	512	0.13	0.4	0.05
10	1 024	0.3	1.1	0.10
12	4 096	1.2	5.6	0.4
14	16 384	5.9	30	1.8
16	65 536	28	200	9
18	262 144	145	>1 000	50

Table 4.4: Time to build the STG and find attractors on synthetic networks (Intel i5-1135G7 @ 2.4 GHz, 16 GB RAM)

Observations:

- **Sync** runs roughly 4–7× faster than **Async** because the number of edges is much smaller (V vs $V \cdot n$).
- **Attractor finding** (Tarjan) accounts for only ~5–10% of the total time; the real bottleneck is the state-space *enumeration*.
- At $n = 16$ sync takes 28s (acceptable); async takes about 3.5 minutes (slow for an interactive UI).
- At $n = 18$ async exceeds 16 minutes – impractical for a web application.

Recommendation: users should run with a full state space up to $n \leq 14$ (sync) or $n \leq 10$ (async). Beyond those limits, switching to *reachable-only* mode – or using PyBoolNet with BDDs – is preferable.

4.5 Bugs found and fixed during development

The implementation process surfaced several bugs worth recording:

1. **Bug #1 – Reserved words and/or/not:** the first version of the parser did not reject variable names that collide with Python keywords. As a result, the rule `or`, `!and` parsed successfully but `eval` raised a `SyntaxError`. Fix: add a `_RESERVED` set and reject those identifiers.
2. **Bug #2 – Async without a self-loop:** an early version of `asynchronous_successors` returned an empty list at fixed points, so Tarjan saw an isolated sink and produced no SCC. Consequence: fixed points were missed in async attractors. Fix: return `[state]` (self-loop) when no diff is found.
3. **Bug #3 – Influence sign for non-monotonic rules:** a rule such as `f = A xor B` (not in the initial tests) caused the `?` sign to be omitted because the `increases` and `decreases` check did not short-circuit correctly. Fix: add an early `return`.
4. **Bug #4 – Tarjan stack overflow in the JS engine:** the first version used a recursive Tarjan implementation; running it on a 2^{16} -state STG hit the JS engine's call-stack limit and threw `RangeError: Maximum call stack size exceeded`. Fix: rewrite Tarjan as an explicit-stack iterative version. After the fix, networks up to $n = 16$ analyse without blowing the stack.

4.6 Experimental summary

- The software runs correctly on all five sample networks, from 2 to 10 nodes.
- Cross-validation against PyBoolNet shows **100% agreement**.
- 23/23 `pytest` cases pass.
- Performance is comfortable for $n \leq 14$; beyond that we recommend reachable-only mode.

5 Conclusion & Future Work

5.1 Summary

The BOOLEAN NETWORK ANALYZER project has implemented **the entire mandatory feature set** required by Topic 6 of the *Programming Integration Project* module:



1. ✓ **.bnet parser**: compatible with the PyBoolNet/BoolNet syntax; correctly handles comments, constants, short/long operators, and word/symbol forms.
2. ✓ **Influence graph**: edges are sign-inferred via a monotonicity test; the graph is rendered with green/red/grey colours.
3. ✓ **State Transition Graph**: both sync and async schemes, full state space or reachable-only.
4. ✓ **Attractor finder**: SCC via Tarjan; fixed-point/cyclic/complex classification; weak-basin computation.
5. ✓ **Visualiser**: clean rendering, attractor highlighting, PNG export for the report.
6. ✓ **Web UI**: zero-install, in-browser web app (vanilla JS + `vis-network`) deployed via GitHub Pages, with five tabs and built-in samples. Live at <https://dangkhoi-dev.github.io/boolean-network-analyzer/>.
7. ✓ **Test & validation**: 23 pytest cases pass; cross-validation against PyBoolNet on five networks shows 100% agreement.

In addition the software offers a few **bonus features**:

- *Trajectory simulation* from a user-chosen initial state.
- Display of each attractor's *basin size*.
- Download of the uploaded `.bnet` file (handy for remixing).
- A friendly warning when the network is large ($n > 12$) before rendering the STG.

Mapping against the rubric:

- *Correctness & Completion (40%)*: full implementation of all mandatory features; cross-validation against PyBoolNet; clear instructions in `README.md`.
- *Technical Report (40%)*: complete Introduction–Theory–Methodology–Experimental–Conclusion structure; figures and tables are captioned and cited.
- *Creativity (10%)*: async support, trajectory simulator, basin size, performance analysis, interactive web UI.
- *Professionalism (10%)*: src-layout, README, requirements.txt, pytest, GitHub Actions, MIT licence.

5.2 Limitations

I openly acknowledge some limitations:

1. **State explosion for $n > 16$:** building the full STG becomes impractical. The current version does not yet support BDD or AIG backends.
2. **Slow UI when rendering STGs with > 1024 nodes:** `matplotlib` layout is slow; at $n = 12$ (4096 nodes) drawing already takes more than 15 seconds and the figure is barely readable.
3. **No multi-valued network (MDD) support:** only Boolean (two-state) networks are handled. Real biological models sometimes require 3–4 levels (e.g. low/medium/high).
4. **No probabilistic updates:** every update is deterministic (sync) or uniformly non-deterministic (async). Probabilistic Boolean Networks (PBNs) are not supported.
5. **No strong basin:** only weak basins are computed.
6. **No persistent cache:** each time the page is reloaded the STG has to be rebuilt from scratch. Using `localStorage` or IndexedDB to cache the last analysis would shave a few hundred milliseconds on warm reloads.

5.3 Future work

Worthwhile extensions include:

1. **Add a BDD/AIG backend** (via `pyeda` or by calling `PyBoolNet`) to scale to $n \geq 50$.
2. **Render the STG with D3 or Plotly** instead of `matplotlib`: interactive zoom/pan, click on a node to expand.
3. **Multi-valued networks (MDD):** extend the parser and engine to support values $\{0, 1, 2, \dots, k\}$; replace Boolean functions with Multi-valued Decision Diagrams.
4. **Probabilistic Boolean Networks (PBN):** allow each node to carry a list of rules with associated probabilities; analyse the resulting Markov chain on the STG.
5. **Control-kernel analysis:** identify the minimum set of nodes whose values must be clamped to steer the system into a desired attractor (used in drug-target identification).



6. **Generate .bnet from the UI:** let the user create the network through a form instead of typing the file by hand.
7. **Automatic PDF report generation:** after analysis, produce a PDF summary with the influence graph, the STG and the attractor table.
8. **Collaborative editing:** multiple users editing the same .bnet on the web via Web-Sockets.

5.4 Acknowledgements

I sincerely thank **Mr. Trinh Van Giang** for his thorough guidance throughout the project. I am also grateful to the PyBoolNet and GINsim communities for releasing their source code and datasets, which were essential for validating my results.

References

- [1] J. D. Schwab, S. D. Kühlwein, N. Ikonomi, M. Kühl, and H. A. Kestler, “Concepts in Boolean network modeling: What do they all mean?”, *Computational and Structural Biotechnology Journal*, vol. 18, pp. 571–582, 2020.
- [2] S. A. Kauffman, “Metabolic stability and epigenesis in randomly constructed genetic nets”, *Journal of Theoretical Biology*, vol. 22, no. 3, pp. 437–467, 1969.
- [3] A. Naldi, D. Berenguier, A. Fauré, F. Lopez, D. Thieffry, and C. Chaouiya, “Logical modelling of regulatory networks with GINsim 2.3”, *Biosystems*, vol. 97, no. 2, pp. 134–139, 2009.
- [4] H. Klarner, A. Bockmayr, and H. Siebert, “PyBoolNet: A Python package for the generation, analysis and visualization of Boolean networks”, *Bioinformatics*, vol. 33, no. 5, pp. 770–772, 2017.
[Online]. Available: <https://github.com/hklarner/pyboolnet>
- [5] R. Thomas, “On the relation between the logical structure of systems and their ability to generate multiple steady states or sustained oscillations”, in *Numerical Methods in the Study of Critical Phenomena*, Springer, Berlin, 1981, pp. 180–193.
- [6] T. S. Gardner, C. R. Cantor, and J. J. Collins, “Construction of a genetic toggle switch in *Escherichia coli*”, *Nature*, vol. 403, no. 6767, pp. 339–342, 2000.
- [7] M. B. Elowitz and S. Leibler, “A synthetic oscillatory network of transcriptional regulators”, *Nature*, vol. 403, no. 6767, pp. 335–338, 2000.
- [8] M. I. Davidich and S. Bornholdt, “Boolean network model predicts cell cycle sequence of fission yeast”, *PLoS ONE*, vol. 3, no. 2, p. e1672, 2008.
- [9] A. Fauré, A. Naldi, C. Chaouiya, and D. Thieffry, “Dynamical analysis of a generic Boolean model for the control of the mammalian cell cycle”, *Bioinformatics*, vol. 22, no. 14, pp. e124–e131, 2006.
- [10] R. E. Tarjan, “Depth-first search and linear graph algorithms”, *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [11] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX”, in *Proceedings of the 7th Python in Science*

Conference (SciPy), Pasadena, CA, USA, 2008, pp. 11–15.

[Online]. Available: <https://networkx.org>

- [12] Almende B.V. and contributors, “vis-network – A dynamic, browser-based visualization library”, 2014–.

[Online]. Available: <https://visjs.github.io/vis-network/>

- [13] J. D. Hunter, “Matplotlib: A 2D graphics environment”, *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

- [14] A. Wuensche, “Discrete dynamics lab: Tools for investigating cellular automata and discrete dynamical networks”, in *Artificial Life Models in Software*, Springer, 2009, pp. 215–258.

- [15] R. Albert and H. G. Othmer, “The topology of the regulatory interactions predicts the expression pattern of the segment polarity genes in *Drosophila melanogaster*”, *Journal of Theoretical Biology*, vol. 223, no. 1, pp. 1–18, 2003.